INFORME DE EXAMEN PRÁCTICO

I. PORTADA

Tema:	Examen Práctico Integrador en C++. Grupo C
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - "B"
Alumnos participantes:	Balarezo Pérez Diego Sebastián
	Cajiao Valdivieso Paulo Alessandro
	Loor Velez Jhon Alejandro

Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. José Caiza

Objetivos:

General:

Desarrollar un módulo de **Ahorcado** en C++ integrado al sistema de inventario del taller tecnológico, que permita al usuario interactuar con palabras relacionadas al dominio del sistema, aplicando estructuras de programación como arreglos, ciclos, validaciones y control de flujo.

Específicos:

- Implementar un menú interactivo para el acceso al juego de Ahorcado.
- Diseñar un sistema de selección de palabras basado en niveles de dificultad.
- Validar correctamente la entrada del usuario, permitiendo solo letras y evitando repeticiones.
- Mostrar el progreso del juego mediante la palabra oculta, letras usadas e intentos restantes.
- Controlar las condiciones de victoria y derrota durante la ejecución del juego.
- Integrar el Ahorcado con el sistema de inventario utilizando términos técnicos del dominio (productos, categorías, movimientos, etc.).
- Registrar resultados y puntuación de las partidas realizadas.

- Incorporar al menos una mejora adicional, como pistas o estadísticas del juego.

Link del GitHub

https://github.com/TurKellou/PruebaUnidad2.git

Pasos Necesarios para Programación en C++

Enunciado:

Ahorcado de rutas, combustible, partes del vehículo y mantenimiento.

Análisis del problema:

Se requiere desarrollar un juego de **Ahorcado** integrado al sistema de inventario del taller tecnológico, en el cual el usuario debe adivinar palabras relacionadas con el dominio del sistema (productos, categorías, movimientos, etc.), ingresando letras de manera controlada.

El sistema debe garantizar una interacción correcta con el usuario, validando entradas y mostrando el progreso del juego.

Implementar un juego interactivo que:

Refuerce el uso de términos del inventario

Permita múltiples partidas

Lleve control de resultados y puntuación

Se integre con el menú principal del sistema

El módulo recibe:

Letras ingresadas por el usuario

Selección de opciones del menú

Nivel de dificultad

Restricciones:

Solo se permiten letras

No se permiten letras repetidas

No se permiten números ni símbolos

Procesos principales

Seleccionar palabra (según dificultad)

Ocultarla con guiones (_ _ _)

Pedir letras al usuario

Verificar si la letra existe en la palabra

Actualizar el progreso

Restar intentos si falla

Terminar cuando:

Adivina la palabra

Se queda sin intentos

Durante el juego:

Palabra parcial (_ _ a _ _)

Letras usadas

Intentos restantes

Al final:

Mensaje de victoria o derrota

Palabra correcta

Puntuación obtenida

El desarrollo del Ahorcado implica resolver varios problemas:

Evitar que el usuario ingrese:

números

símbolos

letras repetidas

Se debe controlar:

número de intentos

letras acertadas

condición de victoria

Se requiere:

almacenar palabras

guardar letras usadas

actualizar palabra oculta

El Ahorcado no debe ser independiente, debe usar:

palabras del inventario

categorías o estados

Estructuras utilizadas

Arreglos → palabras, letras usadas

Ciclos → repetición del juego

Condicionales → validaciones

Funciones → modularidad

Clases (opcional) → organización

El sistema puede incluir:

Pistas según categoría

Sistema de puntuación

Historial de partidas

El módulo de Ahorcado está directamente relacionado con el inventario, ya que:

Utiliza términos técnicos del sistema

Refuerza el conocimiento del dominio

Comparte estructura lógica del programa

Algoritmo paso a paso:

Inicio

Pedir opción del menú del ahorcado

Si elige jugar:

Pedir nivel de dificultad

Seleccionar una palabra según el nivel

Crear una palabra oculta con guiones (_ _ _ _)

Inicializar intentos en 6

Inicializar lista de letras usadas vacía

Proceso

Mostrar la palabra oculta

Mostrar letras usadas

Mostrar intentos restantes

Pedir una letra al usuario

Validar que:

Sea una letra

No esté repetida

Si no cumple → pedir otra letra

Agregar la letra a letras usadas

Buscar la letra en la palabra

Si la letra existe:

Reemplazar guiones por la letra correcta

Si la letra NO existe:

Restar un intento

Verificar si:

La palabra fue completada → gana

Intentos llegaron a 0 → pierde

Repetir hasta que gane o pierda

Salida

Si gana:

Mostrar mensaje “Ganaste”

Si pierde:

Mostrar mensaje “Perdiste”

Mostrar la palabra correcta

Mostrar resultado de la partida

Pseudocódigo

```
// Función Recursiva para calcular intentos (basada en tu código C++)
Funcion res <- CalcularIntentosRecursivo(n)
        Si n = 0 Entonces
                res <- 0
        SiNo
                res <- 1 + CalcularIntentosRecursivo(n-1)
        FinSi
FinFuncion

Algoritmo Proyecto_Ahorcado_Inventario
        // ================= VARIABLES INVENTARIO =================
        // Usamos arreglos paralelos para simular el objeto 'Producto'
        Dimension nombres[50], categorias[50], stocks[50], precios[50]
        total <- 0

        Repetir
                Escribir ""
                Escribir "--- MENU PRINCIPAL ---"
```

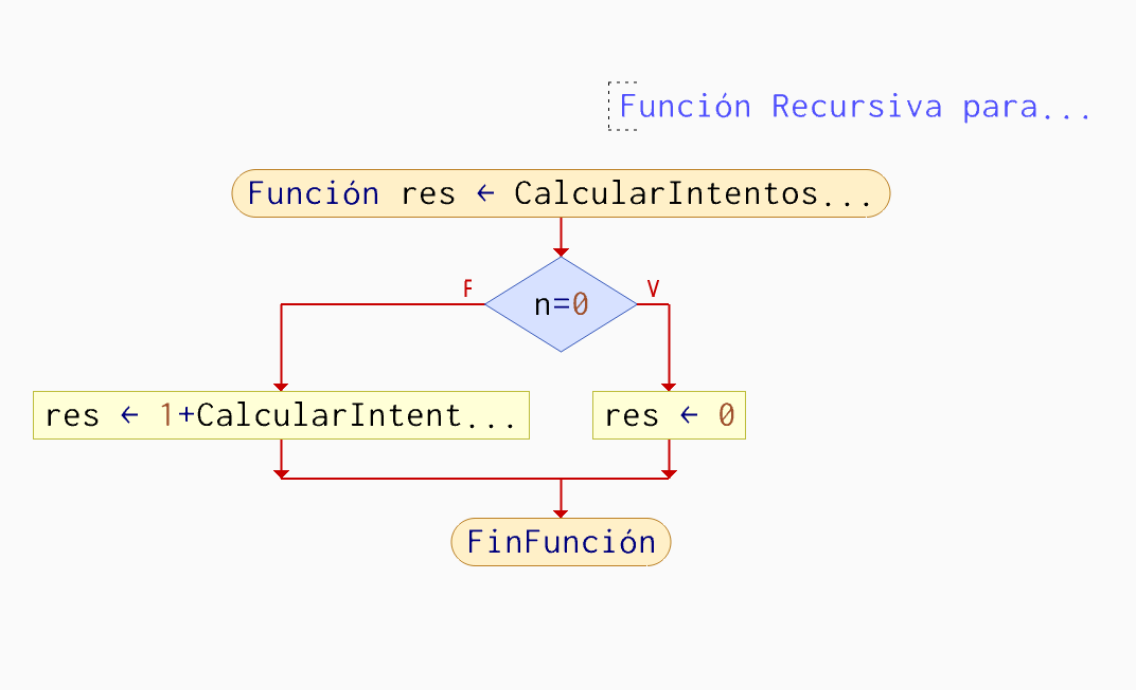
```
Escribir "1. Inventario"
Escribir "2. Ahorcado"
Escribir "3. Mostrar intentos (recursivo)"
Escribir "4. Salir"
Escribir "Seleccione una opcion:"
Leer opcion

Segun opcion Hacer
    1:
        // Submenú Inventario
        Repetir
            Escribir ""
            Escribir "============ INVENTARIO ============"
            Escribir "1. Agregar"
            Escribir "2. Mostrar"
            Escribir "3. Buscar"
            Escribir "4. Ordenar (Intercambiar)"
            Escribir "5. Volver"
            Escribir "Opcion:"
            Leer op2

            Segun op2 Hacer
                1:
                    Si total < 50 Entonces
                        total <- total + 1
                        Escribir "Nombre del producto:"
                        Leer nombres[total]
                        Escribir "Categoria:"
                        Leer categorias[total]
                        Escribir "Stock:"
                        Leer stocks[total]
```

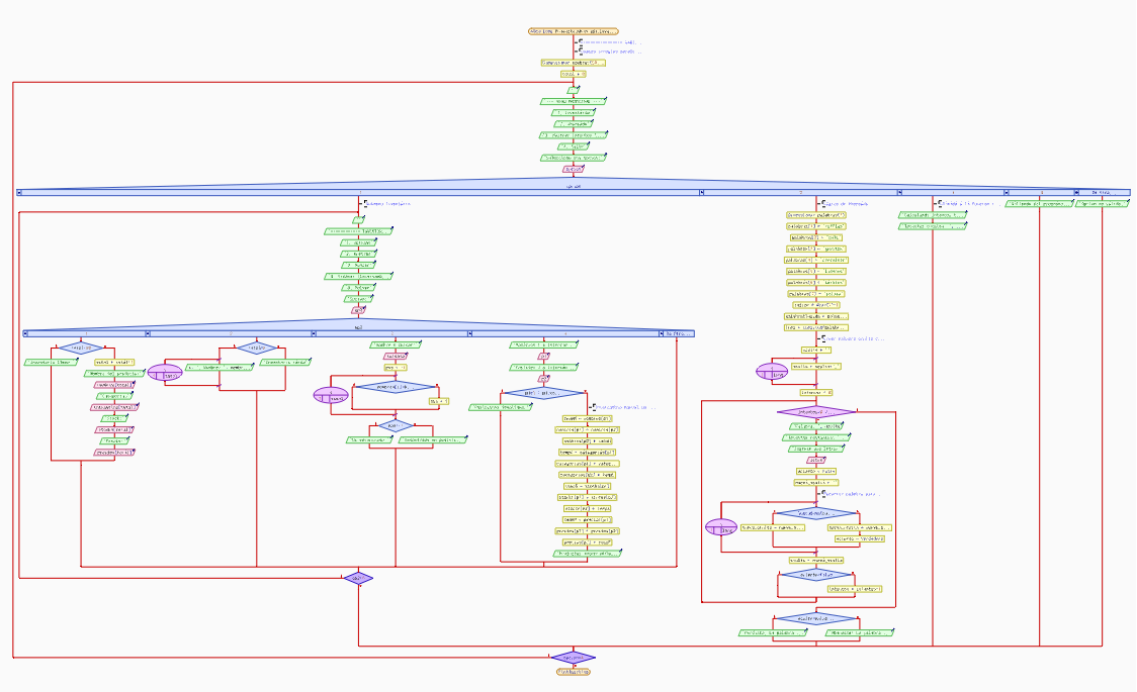
```
            Escribir "Precio:"
            Leer precios[total]
        SiNo
            Escribir "Inventario lleno."
        FinSi
2:
        Si total = 0 Entonces
            Escribir "Inventario vacio"
        SiNo
            Para i <- 1 Hasta total Hacer
                Escribir i, ". Nombre: ", nombres[i], " | Cat: ", categorias[i], " | Stock: ", stocks[i], " | Precio: $", precios[i]
            FinPara
        FinSi
3:
        Escribir "Nombre a buscar:"
        Leer buscado
        pos <- -1
        Para i <- 1 Hasta total Hacer
            Si nombres[i] = buscado Entonces
                pos <- i
            FinSi
        FinPara
        Si pos <> -1 Entonces
            Escribir "Encontrado en posicion: ", pos
        SiNo
            Escribir "No encontrado."
        FinSi
4:
```

```
                    Escribir "Posicion 1 a intercambiar:"
                    Leer p1
                    Escribir "Posicion 2 a intercambiar:"
                    Leer p2
                    Si p1 >= 1 Y p1 <= total Y p2 >= 1 Y p2 <= total Entonces
                        // Intercambio manual de cada atributo
                        tempN <- nombres[p1]; nombres[p1] <- nombres[p2]; nombres[p2] <- tempN
                        tempC <- categorias[p1]; categorias[p1] <- categorias[p2]; categorias[p2] <- tempC
                        tempS <- stocks[p1]; stocks[p1] <- stocks[p2]; stocks[p2] <- tempS
                        tempP <- precios[p1]; precios[p1] <- precios[p2]; precios[p2] <- tempP
                        Escribir "Productos intercambiados correctamente."
                    SiNo
                        Escribir "Posiciones invalidas."
                    FinSi
                FinSegun
            Hasta Que op2 = 5

        2:
            // Lógica de Ahorcado
            Dimension palabras[7]
            palabras[1] <- "ruffles"; palabras[2] <- "dell"; palabras[3] <- "gomitas"
```

```
palabras[4] <- "chocolate"; palabras[5] <- "limones"; palabras[6] <- "barbies"; palabras[7] <- "polaca"

indice <- Azar(7) + 1
palabraElegida <- palabras[indice]
long <- Longitud(palabraElegida)

// Crear palabra oculta con guiones
oculto <- ""
Para i <- 1 Hasta long Hacer
    oculto <- oculto + "_"
FinPara

intentos <- 6
Mientras intentos > 0 Y oculto <> palabraElegida Hacer
    Escribir "Palabra: ", oculto
    Escribir "Intentos restantes: ", intentos
    Escribir "Ingrese una letra:"
    Leer letra

    acierto <- Falso
    nueva_oculta <- ""

    // Recorrer palabra para verificar letra
    Para i <- 1 Hasta long Hacer
        Si Subcadena(palabraElegida, i, i) = letra Entonces
            nueva_oculta <- nueva_oculta + letra
            acierto <- Verdadero
        SiNo
            nueva_oculta <- nueva_oculta + Subcadena(oculto, i, i)
```

```
                    FinSi
                FinPara

                oculto <- nueva_oculta

                Si acierto = Falso Entonces
                    intentos <- intentos - 1
                FinSi
            FinMientras

            Si oculto = palabraElegida Entonces
                Escribir "¡Ganaste! La palabra era: ", palabraElegida
            SiNo
                Escribir "Perdiste. La palabra era: ", palabraElegida
            FinSi

        3:
            // Llamada a la función recursiva
            Escribir "Calculando intentos totales mediante recursividad..."
            Escribir "Intentos totales: ", CalcularIntentosRecursivo(6)

        4:
            Escribir "Saliendo del programa..."

        De Otro Modo:
            Escribir "Opcion no valida."
    FinSegun
Hasta Que opcion = 4

FinAlgoritmo
```





Codificación en C++

```
#include <iostream>

#include <string>
```

```
#include <cstdlib>

#include <ctime>

using namespace std;

// ================= CLASE PRODUCTO =================

class Producto {

private:

string nombre;

string categoria;

int stock;

float precio;

public:

Producto(string n = "", string c = "", int s = 0, float p = 0) {

nombre = n;

categoria = c;

stock = s;

precio = p;

}

string getNombre() { return nombre; }

string getCategoria() { return categoria; }

int getStock() { return stock; }

float getPrecio() { return precio; }
```

```
void setStock(int s) {
if (s >= 0) stock = s;
}
void setPrecio(float p) {
if (p >= 0) precio = p;
}
void mostrar() {
cout << "Nombre: " << nombre << endl;
cout << "Categoria: " << categoria << endl;
cout << "Stock: " << stock << endl;
cout << "Precio: " << precio << endl;
cout << "----------------------" << endl;
}
};
// ================= CLASE INVENTARIO =================
class Inventario {
private:
Producto productos[50];
int total;
public:
Inventario() { total = 0; }
```

```
void agregarProducto() {

string nombre, categoria;

int stock;

float precio;

cout << "Nombre: " << endl;

cin >> nombre;

cout << "Categoria: " << endl;

cin >> categoria;

cout << "Stock: " << endl;

while (!(cin >> stock) || stock < 0) {

cin.clear();

cin.ignore(1000, '\n');

cout << "Dato invalido. Intente de nuevo: " << endl;

}

cout << "Precio: " << endl;

while (!(cin >> precio) || precio < 0) {

cin.clear();

cin.ignore(1000, '\n');

cout << "Dato invalido. Intente de nuevo: " << endl;

}

productos[total] = Producto(nombre, categoria, stock, precio);
```

```
total++;
}
void mostrar() {
if (total == 0) {
cout << "Inventario vacio" << endl;
return;
}
for (int i = 0; i < total; i++) {
cout << i + 1 << "." << endl;
productos[i].mostrar();
}
}
int buscar(string nombre) {
for (int i = 0; i < total; i++) {
if (productos[i].getNombre() == nombre)
return i;
}
return -1;
}
void ordenar(int pos1, int pos2) {
if (pos1 >= 0 && pos1 < total && pos2 >= 0 && pos2 < total) {
```

```
Producto temp = productos[pos1];

productos[pos1] = productos[pos2];

productos[pos2] = temp;

cout << "Productos intercambiados correctamente" << endl;

} else {

cout << "Posiciones invalidas" << endl;

}

}

};

// RECURSIVIDAD

int intentosRestantes(int n) {

if (n == 0)

return 0;

return 1 + intentosRestantes(n - 1);

}

// AHORCADO

void Ahorcado() {

srand(time(0));

int min = 0;

int max = 6;

int aleatorio = rand() % (max - min + 1) + min;
```

```
string palabra[7] = {"ruffles", "dell", "gomitas", "chocolate", "limones", "barbies", "polaca"};

string oculto(palabra[aleatorio].length(), '_');

int intentos = 6;

char letra;

while (intentos > 0 && oculto != palabra[aleatorio]) {

cout << "Palabra: " << oculto << endl;

cout << "Intentos: " << intentos << endl;

cout << "Letra: ";

cin >> letra;

bool acierto = false;

for (int i = 0; i < palabra[aleatorio].length(); i++) {

if (palabra[aleatorio][i] == letra) {

oculto[i] = letra;

acierto = true;

}

}

if (!acierto)

intentos--;

}

if (oculto == palabra[aleatorio])

cout << "Ganaste!\n";
```

```
else

cout << "Perdiste. Era: " << palabra[aleatorio] << endl;

}

// MAIN

int main() {

Inventario inv;

int opcion;

do {

cout << "--- MENU PRINCIPAL ---" << endl;

cout << "1. Inventario" << endl;

cout << "2. Ahorcado" << endl;

cout << "3. Mostrar intentos (recursivo)" << endl;

cout << "4. Salir" << endl;

cout << "Opcion: " << endl;

while (!(cin >> opcion)) {

cin.clear();

cin.ignore(1000, '\n');

cout << "Dato invalido: " << endl;

}

switch (opcion) {

case 1: {
```

```
int op2;

do {

cout << "============ INVENTARIO ============" << endl;

cout << "1. Agregar" << endl;

cout << "2. Mostrar" << endl;

cout << "3. Buscar" << endl;

cout << "4. Ordenar" << endl;

cout << "5. Volver" << endl;

cout << "Opcion: " << endl;

while (!(cin >> op2)) {

cin.clear();

cin.ignore(1000, '\n');

cout << "Opcion invalida: " << endl;

}

switch (op2) {

case 1:

inv.agregarProducto();

break;

case 2:

inv.mostrar();

break;
```

```
case 3: {
string nombre;
cout << "Buscar: " << endl;
cin >> nombre;
int pos = inv.buscar(nombre);
if (pos != -1)
cout << "Encontrado en posicion " << pos + 1 << endl;
else
cout << "No encontrado" << endl;
break;
}
case 4: {
int p1, p2;
cout << "Posicion 1: " << endl;
cin >> p1;
cout << "Posicion 2: " << endl;
cin >> p2;
inv.ordenar(p1 - 1, p2 - 1);
break;
}
case 5:
```

```
cout << "Volviendo..." << endl;

break;

default:

cout << "Opcion invalida" << endl;

}

} while (op2 != 5);

break;

}

case 2:

Ahorcado();

break;

case 3:

cout << "Intentos totales: " << intentosRestantes(6) << endl;

break;

case 4:

cout << "Saliendo..." << endl;

break;

default:

cout << "Opcion invalida" << endl;

}

} while (opcion != 4);
```

```
return 0;

}
```

Resolución del Fragmento de código defectuoso

Fragmento Original:

```
float depreciar(float precio, int meses, float tasa){
   for(int i=1; i<meses; i++)
      precio = precio - precio * tasa / 100;
   if(precio <= 0)
      precio == 0.01;
   return tasa;
}
```

Resolución:

```
float depreciar(float precio, int meses, float tasa){
   for(int i = 0; i < meses; i++){
      precio = precio - (precio * tasa / 100);
   }

   if(precio <= 0){
      precio = 0.01;
   }

   return precio;
}
```

Prueba de escritorio:

```
47   ventario {
104  | ordenar(int pos1, int pos2) {
110   } else {
112   }
113
114
115
116  ISIVIDAD
117  ntosRestantes(int n) {
118  n == 0)
119  return 0;
120  rn 1 + intentosRestantes(n - 1);
121
122
123  ADO
124  rcado() {
125  d(time(0));
126  min = 0;
127  max = 6;
128  aleatorio = rand() % (max - min + 1) + min;
129
130  ng palabra[7] = {"ruffles", "dell", "gomitas", "chocolate", "limones", "barbies", "pola
131  ng oculto(palabra[aleatorio].length(), '_');
132  intentos = 6;
133  letra;
134
135  e (intentos > 0 && oculto != palabra[aleatorio]) {
136  cout << "Palabra: " << oculto << endl;
137  cout << "Intentos: " << intentos << endl;
138  cout << "Letra: ";
139  cin >> letra;
140
141  bool acierto = false;
142
143
144  for (int i = 0; i < palabra[aleatorio].length(); i++) {
145      if (palabra[aleatorio][i] == letra) {
146          oculto[i] = letra;
147          acierto = true;
148      }
149  }
```

File Edit View Repository Branch Help

Current repository: PruebaUnidad2 | Current branch: main | Fetch origin — Last fetched 7 minutes ago

Changes | History

No branches to compare

- Merge branch 'main' of https://github.... — TurKellou • 7 minutes ago
- Version v1.4 — 3 people • 8 minutes ago
- Update README.md — Paulo Cajiao • 12 minutes ago
- Update README.md — Paulo Cajiao • 12 minutes ago
- Version v1.3.1 — 3 people • 15 minutes ago
- Version v1.3 — 3 people • 18 minutes ago
- Version v1.2 — 3 people • 1 hour ago
- Versión Base — 3 people • 1 hour ago
- Version Base — TurKellou • 1 hour ago
- Create PruebaUnidad2.cpp — TurKellou • 1 hour ago
- Initial commit — TurKellou • 1 hour ago

Version v1.3

Co-Authored-By: JhonLoor <265168561+jhon-loor@users.noreply.github.com>
Co-Authored-By: Diego Balarezo <265016553+diegosw-08@users.noreply.github.com>

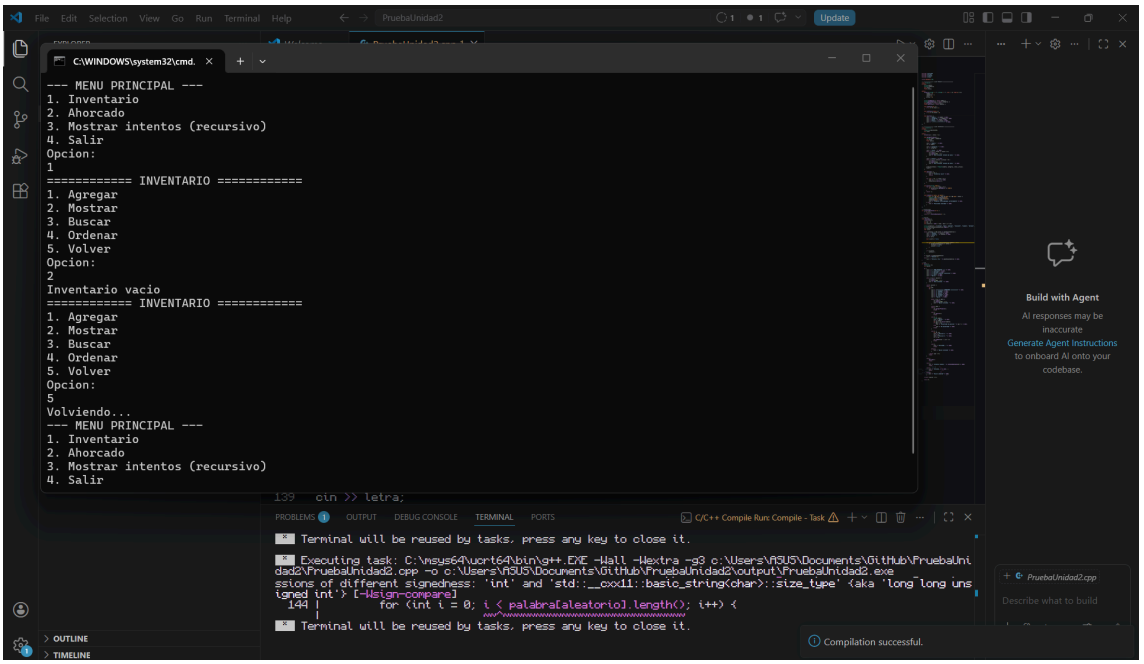
3 people — 32ab306

2 changed files

- PruebaUnidad2.cpp
- output\PruebaUnidad2.exe

PruebaUnidad2.cpp

```diff
@@ -2,35 +2,41 @@
 2  2  #include <string>
 3  3  using namespace std;
 4  4
 5     - //Clase producto
    5  + // ================ CLASE PRODUCTO ================
 6  6  class Producto {
 7  7  private:
 8  8      string nombre;
 9  9      string categoria;
10 10      int stock;
   11  +   float precio;
11 12
12 13  public:
13    -     // Constructor
14    -     Producto(string n = "", string c = "", int s = 0) {
   14  +     Producto(string n = "", string c = "", int s = 0, float p = 0) {
15 15          nombre = n;
16 16          categoria = c;
17 17          stock = s;
   18  +       precio = p;
18 19      }
19 20
20    -     // Métodos get y set (encapsulamiento)
21 21      string getNombre() { return nombre; }
22 22      string getCategoria() { return categoria; }
23 23      int getStock() { return stock; }
   24  +   float getPrecio() { return precio; }
24 25
25 26      void setStock(int s) {
26    -         if (s >= 0)
27    -             stock = s;
```



Conclusiones:

El código del juego del ahorcado permite comprender de manera práctica cómo se aplican los conceptos básicos de programación en C++, como el uso de estructuras de control, variables y manejo de cadenas de texto para construir un programa interactivo. A través de su desarrollo se evidencia la importancia de la lógica para controlar los intentos del jugador, verificar las letras ingresadas y mostrar el progreso del juego. Además, este tipo de proyecto fortalece el pensamiento lógico y la capacidad de resolver problemas, ya que se debe organizar correctamente cada parte del programa para que funcione de forma adecuada. En general, el juego del ahorcado demuestra cómo la programación puede utilizarse para crear aplicaciones dinámicas y también deja ver que siempre existen posibilidades de mejora y optimización en el código.